

Phase -1.7 — API Design & REST Contract

Objective

Before implementing a single endpoint, the entire API contract should be designed.

This ensures that:

- Frontend and backend teams can work independently.
- API changes are intentional rather than accidental.
- Request and response formats remain consistent.
- Validation rules are clearly defined.
- Error handling is standardized.
- Documentation can be generated automatically.

The API contract is considered the agreement between the client and the server.

API Design Principles

The APIs in this project will follow RESTful principles.

Resource-Oriented Design

Endpoints should represent **resources**, not actions.

Good

```
GET    /events
POST   /bookings
GET    /users/me
```

Avoid

```
/createBooking
/getEvents
/deleteSeat
```

Resources should be represented using nouns.

HTTP methods describe the action.

HTTP Methods

GET

Retrieve resources.

Must never modify server state.

Examples

```
GET /events
GET /events/{id}
GET /events/{id}/seats
```

POST

Create a new resource.

Examples

```
POST /auth/register
POST /auth/login
POST /bookings
POST /payments
```

PUT

Replace an existing resource.

Future use:

```
PUT /events/{id}
```

PATCH

Partially update an existing resource.

Future use:

```
PATCH /users/me
```

DELETE

Remove a resource. (Soft delete Only)

Example

```
DELETE /events/{id}
```

API Versioning

The application should support API versioning from the beginning.

Version 1

```
/api/v1/events
```

Future versions

```
/api/v2/events
```

Although only Version 1 will exist initially, designing with versioning avoids breaking future clients.

Endpoint Naming Convention

General format

```
/api/v1/{resource}  
/api/v1/{resource}/{id}  
/api/v1/{resource}/{id}/{child-resource}
```

Examples

```
/api/v1/events  
/api/v1/events/{id}  
/api/v1/events/{id}/seats  
/api/v1/bookings  
/api/v1/auth/login
```

Authentication APIs

Register

```
POST /api/v1/auth/register
```

Purpose

Creates a new user account.

Authentication Required

No

Login

```
POST /api/v1/auth/login
```

Purpose

Authenticates a user and returns tokens.

Authentication Required

No

Refresh Token

```
POST /api/v1/auth/refresh
```

Purpose

Rotates refresh tokens and issues a new access token.

Authentication Required

No

Logout

```
POST /api/v1/auth/logout
```

Purpose

Revokes the current refresh token.

Authentication Required

Yes

User APIs

Current User

```
GET /api/v1/users/me
```

Purpose

Returns information about the authenticated user.

Booking History

```
GET /api/v1/users/me/bookings
```

Purpose

Returns all bookings made by the authenticated user.

Event APIs

List Events

GET /api/v1/events

Seat Layout

GET /api/v1/events/{eventId}/seats

Create Event

POST /api/v1/events

Administrator only.

Update Event

PUT /api/v1/events/{eventId}

Administrator only.

Delete Event

DELETE /api/v1/events/{eventId}

Administrator only.

Booking APIs

Create Booking

```
POST /api/v1/bookings
```

Creates a temporary booking.

Get Booking

```
GET /api/v1/bookings/{bookingId}
```

Returns booking information.

Payment APIs

Process Payment

```
POST /api/v1/payments
```

Processes payment for a booking.

Payment Status

```
GET /api/v1/payments/{paymentId}
```

Returns payment information.

Administration APIs

Future administrator APIs may include

```
GET /api/v1/admin/bookings  
  
GET /api/v1/admin/statistics  
  
GET /api/v1/admin/events  
  
GET /api/v1/admin/users
```

Health APIs

```
GET /health
```

Purpose

Infrastructure health checks.

```
GET /metrics
```

Purpose

Prometheus metrics.

Request Structure

Every request should follow the same format.

Headers

```
Authorization: Bearer <access_token>  
  
Content-Type: application/json
```

Path Parameters

```
/events/{eventId}
```


Query Parameters

```
?page=1

&limit=20

&sort=startTime

&order=asc
```

Body

```
{
  "email": "user@example.com",
  "password": "password123"
}
```

Standard Success Response

Every successful response should follow a consistent structure.

```
{
  "success": true,
  "data": {},
  "timestamp": "2026-07-05T10:45:22Z",
  "requestId": "req_01JABCXYZ"
}
```

This consistency simplifies frontend development and debugging.

Standard Error Response

Every error should follow the same format.

```
{
  "success": false,
  "error": {
    "code": "SEAT_ALREADY_BOOKED",
  }
}
```

```
    "message": "The selected seat is no longer available.",
  },
  "timestamp": "2026-07-05T10:45:22Z",
  "requestId": "req_01JABCXYZ"
}
```

All errors should be produced by the Global Error Handler.

HTTP Status Codes

The application will use standard HTTP status codes consistently.

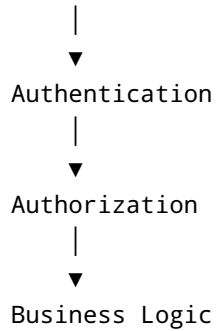
Status	Meaning	Usage
200	OK	Successful GET
201	Created	Resource created
204	No Content	Successful delete
400	Bad Request	Validation failure
401	Unauthorized	Missing or invalid authentication
403	Forbidden	Insufficient permissions
404	Not Found	Resource not found
409	Conflict	Seat already booked
422	Unprocessable Entity	Business rule violation
429	Too Many Requests	Rate limiting
500	Internal Server Error	Unexpected failure

Status codes should be meaningful and never overloaded.

Validation Strategy

Every request passes through three stages.

```
Client
|
▼
Request Validation
```



Invalid requests should fail as early as possible.

Idempotency

Some endpoints should be idempotent.

Examples

GET
PUT
DELETE

Booking and payment endpoints are intentionally **not idempotent** because they create new resources.

Future payment integrations may introduce idempotency keys.

Pagination Strategy

Large collections should support pagination.

Example

GET /events?page=1&limit=20

Future enhancements

- Cursor pagination
- Filtering

- Search
 - Sorting
-

API Documentation

The OpenAPI specification will become the single source of truth.

Documentation should include:

- Endpoints
- Parameters
- Request schemas
- Response schemas
- Authentication requirements
- Example payloads
- Error responses

Swagger UI will be generated directly from this specification during implementation.

Deliverables

At the completion of this phase, the entire external contract of the application is finalized.

Frontend developers can begin implementation without backend code.

Backend developers know:

- Every endpoint.
- Every HTTP method.
- Every request format.
- Every response format.
- Every status code.
- Every validation rule.
- Every authentication requirement.

The next phase, **Phase -1.8 — Database Design & Entity Relationship Diagram (ERD)**, will define the domain model, entities, relationships, constraints, and database normalization strategy before any tables are created.